PromiseTracker v3 system specification

Version 3.0, draft, 12 November 2026.

This is the full specification for PromiseTracker v3: what the system does, how it is built, and the detail a developer needs to build it. Each section opens with a short summary in plain language, so a reader who is not a developer can follow the design and skip the detail.

The product specification explains the same system without the technical detail.

Contents

1. Overview
2. Key terms
3. How the system is put together
4. Plugins and ports
5. Verification plugins
6. Other plugins
7. Data model
8. Publishing rules
9. The pipeline
10. The UI layer
11. Web addresses
12. The API
13. Security
14. Errors and logging
15. Testing
16. Deployment and operations
17. Performance
18. Measures
19. Development and delivery
20. Decisions
21. Appendix A: items to confirm
22. Appendix B: what v2 does today

The word CONFIRM marks a statement that someone must check against a running system before building on it. Appendix A lists every one.

1. Overview

1.1 What the system does

PromiseTracker records what people in power promise and claim, checks each claim against its source, and follows what happens to it over time. Editors bring in source documents. AI proposes candidate claims and a person reviews every one. A fact-checker verifies each claim. Editors rate it, publish it, and add dated updates as events unfold. The public reads the result on a website.

Visitors can send evidence and corrections, but an editor decides what changes the record. That is the difference from trackers that let politicians report on themselves or let the public vote.

1.2 What is new in v3

Version 2 cannot run without Airtable and Meedan Check, has no record of how a status changed, and has a public site that does not cache, paginate or search. Appendix B lists the evidence.

Version 3 changes these things:

- **The core runs alone,** on PostgreSQL and file storage. Every outside service is optional.
- **Outside services are plugins** behind fixed interfaces, so one can be swapped for another.
- **The public site is a replaceable layer.** It ships with the platform but reads everything through a typed client.
- **One site serves every country,** with the language and country in the web address.
- **Claims have a history.** Each claim carries dated updates with evidence, and its current rating comes from the latest one.
- **Editors define the ratings.** Six ratings in three tiers ship as the default.
- **Location is a way in.** Each country has its own lists of offices and areas, so a search for a place returns everyone who represents it.

Version 2 never launched, so there is no data to migrate.

1.3 Scope

In scope:

- Countries, languages, areas, offices, actors and their terms, claims of any type, updates, ratings, documents, topics, themes, articles, citizen submissions and media
- Ingestion from uploads, links and spreadsheets, with AI-assisted extraction and human review
- Verification in the built-in review queue or Meedan Check, and later Superdesk
- A public site with country, actor, claim, area, topic, theme and search pages
- A typed API that the public site uses, opened to outside users later
- English and French at launch, with more languages added by editors

Out of scope for now, with room left in the design:

Left out	How it can be added later
Editing rights per country	Each user already has a country list that is not yet enforced. Enforcing it changes no stored data
A separate web address or theme per country	Routing reads the country from the path, and themes are tokens
Public user accounts, alerts, petitions	Submissions are anonymous today
Detecting claims in the news automatically	A new source connector could feed it
Verification by the public	Citizens send evidence. A fact-checker decides
A dedicated search engine	Search sits behind a port

1.4 Technology

Layer	Choice
Content management and admin	Payload 4, pinned to an exact beta until 4.0 is released
Database	PostgreSQL, through Payload's Postgres adapter
Web framework	Next.js 16.4 or later with the App Router, React 19
Language	TypeScript 6 in strict mode, Node 24.22.1 or later
File storage	Any S3-compatible store
Document reading	Apache Tika, self-hosted
AI	Vercel AI SDK, behind the LLM provider port
Background jobs	Payload Jobs
Styling of the public site	Tailwind CSS 4 with theme tokens
Error tracking	Sentry
Tests	Vitest and Playwright
Packaging	One Docker image
Licence	MIT

1.5 Rules the design must keep

1. The system runs with no outside service. With only the core plugin enabled, on Postgres and file storage, a team can bring in documents, enter claims, verify, rate, publish and track them. Automated tests run in this configuration.
2. Nothing reaches the public without human review.
3. Nobody types in a claim's current rating. The system derives it from the latest published update.
4. The server never fetches a web address a user supplied, with one audited exception in section 13.4.
5. One field records whether a record should be public. The system computes whether it is, for each language.

6. No sync, import or plugin deletes a record. A deletion upstream archives the record here.
7. The public site never talks to Payload directly. It goes through the SDK.

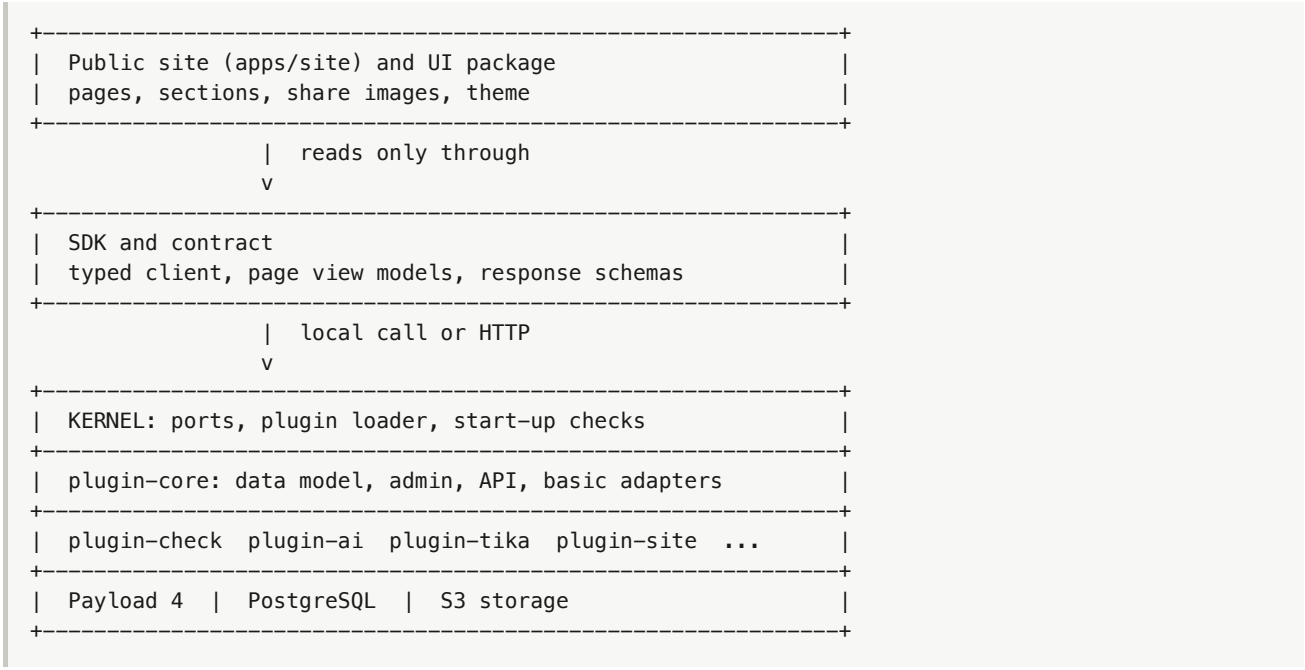
2. Key terms

Term	Meaning
Actor	A person, party or institution that makes claims
Area	A place in a country's nested list of places, such as a county or constituency
Office	A title from a country's list, such as Governor or MP
Seat	An area plus an office, such as Nairobi County plus Governor
Term	An actor's time in a seat, with start and end dates
Claim	A promise or other checkable statement. A promise is the default claim type
Claim type	A kind of claim, defined by editors. Each type uses one rating scale
Rating scale	A named set of tiers and ratings. The default scale is "Delivery"
Rating	Where a claim stands on its scale, such as In Progress
Tier	A group of ratings shown where space is short, such as Promise kept
Claim update	A dated entry in a claim's history: the new rating, what happened, and the evidence
Verdict	The fact-checker's finding on whether the claim is accurately quoted and attributed. Separate from the rating
Record status	The setting an editor controls on each record: ready, hold or archived
Published status	Whether a record is live in a given language. The system computes it
Plugin	A self-contained package that adds features
Port	A fixed interface for one job, such as "verify a claim"
Adapter	A plugin's implementation of a port
Kernel	The base layer that defines the ports and loads the plugins
Core	The plugin that holds the data model and a basic adapter for every port
SDK	The typed client the public site uses to read data

3. How the system is put together

In short: a small kernel defines the interfaces. The core plugin holds everything PromiseTracker needs to work alone. Other plugins connect outside services. The public site sits on top and reads data only through the SDK.

3.1 The layers



The kernel defines the ports, loads plugins and checks the configuration at start-up. It builds on Payload's `definePlugin` and adds what Payload lacks: ports, the rule that each required port has exactly one adapter, and field ownership.

The core plugin owns the data model, the admin screens and the API, and ships a basic adapter for every port.

Integration plugins each connect one outside service. Each depends on the core and never calls another plugin directly.

plugin-site holds the settings that only the default public site uses: homepage and country picks, section overrides, explainer text, navigation and interface strings. A replacement site brings its own settings plugin, so the core stays free of UI concerns.

The public site is described in section 10.

3.2 Why the core is a plugin too

If the core were the application and plugins were bolted on, each integration would reach into its internals. That is how v2 came to write Airtable status messages from six places. Making the core a plugin means it follows the same rules as everything else, and weak rules show up early.

3.3 Supported configurations

The `PT_PLUGINS` environment variable lists the enabled plugins.

Enabled plugins	Document reading	Candidates	Verification
core, site	Plain text and HTML	Entered by hand	Built-in review queue
core, site, tika	Adds PDF, Word, Excel and more	Entered by hand	Built-in review queue
core, site, tika, ai	Same	AI proposes, a person reviews	Built-in review queue
core, site, tika, ai, check	Same	Same	Meedan Check. This is the version 1 set-up
core, site, tika, ai, superdesk	Same	Same	Superdesk, in a later version

4. Plugins and ports

In short: a port describes a job and an adapter does it. Start-up checks make sure every required job has exactly one adapter. Ownership rules make sure two systems never write the same field.

4.1 The ports

Port	The job	Adapters that run	In core	Others
VerificationProvider	Where people verify claims	Exactly one	`native`	`check`, `superdesk`
SourceConnector	Where content comes from	Any number. Upload is always on	`upload`, `url`	`airtable`, `sheets`, `feed`
ContentExtractor	Turning a file into text	One or more, by file type	`plaintext`	`tika`, `ocr`
StructuredImporter	Turning spreadsheet rows into records	Any number	`csv`, `xlsx`	`sheets`
LLMProvider	Which AI model proposes candidates	Many installed, one selected	None	`google`, `openai`, `anthropic` and others
SearchProvider	Full-text and place search	Exactly one	`postgres`	A dedicated search engine
NewsletterProvider	Newsletter sign-ups	Exactly one	`local`, which stores confirmed sign-ups	Mailing-list services
AnalyticsProvider	Counting visits and searches	Exactly one	`none`	Privacy-respecting analytics tools
CaptchaProvider	Stopping bots on public forms	Exactly one	`local`: a proof-of-work check, a hidden field and rate limits	Turnstile, hCaptcha

Database, file storage, email and the admin's web framework are replaced through Payload's own adapters rather than ports.

Ingestion uses three ports because the inputs differ. A spreadsheet of promises maps straight to records. A manifesto needs reading, then AI, then review.

4.2 What each port provides

Every adapter provides a `health` method that reports whether its service is reachable.

Port	Methods
VerificationProvider	`submit(claim)` sends a claim out and returns the outside reference. `pull(since)` fetches verdicts changed since a saved position. `receive(payload, headers)` handles a push, if the service sends them
SourceConnector	`mode` is manual, pull or push. `list(since)` lists items waiting, for pull connectors. `fetch(item)` retrieves one
ContentExtractor	`accepts` lists file types, most specific wins. `extract(content)` returns text, a page count and any metadata
StructuredImporter	`accepts` lists file types. `inspect(content)` reads headings and sample rows without saving. `import(content, mapping)` creates or updates records
LLMProvider	`models` lists the models. `generate(request)` returns structured output
SearchProvider	`search(query, scope)` returns grouped results. `resolvePlace(text)` returns matching areas. `index(record)` keeps the index current
NewsletterProvider	`subscribe(email, locale)` starts a confirmed sign-up. `unsubscribe(email)`
AnalyticsProvider	`pageView(event)` and `searchEvent(event)`. Neither receives personal data
CaptchaProvider	`challenge()` returns what the form shows. `verify(token, request)` returns pass or fail

Each method receives a context object with the database handle, a logger, a cancel signal and the plugin's own settings. An adapter never reads environment variables itself and never reads another plugin's settings.

4.3 What a plugin declares

A plugin is defined with Payload's `definePlugin`, with a slug and an order, and adds a manifest.

Manifest entry	Purpose
`slug`, `version`, `order`	Identity and load order
`dependsOn`	Plugins that must be present
`collections`, `globals`	Record types the plugin owns
`extend`	Fields added to record types another plugin owns, section 4.5
`provides`	Adapters, by port
`tasks`, `workflows`	Background jobs
`routes`	Web routes it serves, section 4.7
`settings`	A settings group under the plugin's slug
`widgets`	Admin dashboard cards
`requiredEnv`	Environment variables it needs

Three environment variables select the set-up:

```
PT_PLUGINS=core,site,tika,ai,check
PT_VERIFICATION=check
PT_LLM_MODEL=google:gemini-2.5-pro
```

4.4 What happens at start-up

1. Read `PT_PLUGINS` and load the manifests.
2. Check dependencies and load order. A circular dependency stops start-up.
3. For each port that needs exactly one adapter, find it. None, or more than one, stops start-up with a message naming the candidates and the variable that chooses.
4. Register every first-party plugin's record types and fields, enabled or not. See section 4.6.
5. Wire hooks, routes, jobs and widgets for enabled plugins only.
6. Check required environment variables and the Postgres extensions in section 16.2.
7. Run each enabled plugin's own settings check, such as the Check plugin's status mapping.
8. Log the final configuration in one line.

A deployment with nowhere to verify claims is broken, and it should fail when deployed, not when a fact-checker first opens a claim.

4.5 Ownership rules

1. Fields a plugin adds are always optional, so removing the plugin leaves valid records.
2. A plugin may not change the type of a core field.
3. A field a plugin owns is read-only in the admin. Only the plugin and a super-admin can write it, and the system logs a super-admin override.
4. A plugin's fields sit in a group named after it, such as `check.itemId`.
5. A plugin never deletes a core record. It archives it.

When a plugin owns a job, the matching admin role loses it.

Plugin enabled	What it takes over	Effect in the admin
`plugin-check` or `plugin-superdesk`	The verdict, verification date and rationale	Reviewers can no longer set a verdict. They still review AI candidates
`plugin-airtable`	Intake of documents and actors	Contributors can no longer create those records

4.6 Plugins and the database schema

With Postgres, every field is a column and every change needs a migration. Payload keeps one migration history per project. So the schema always includes the record types and fields of every first-party plugin, whether it is enabled or not. `PT_PLUGINS` decides behaviour, routes, ports and admin visibility, not the schema.

A disabled plugin's fields stay in the database, hidden from the admin and never written. A third-party plugin added by one deployment ships its own migrations.

4.7 Plugin routes

The web framework builds its routes from files on disk, so plugin routes live in a fixed folder and are listed in each manifest.

```
src/app/api/plugins/
  check/webhook/route.ts       POST  report events from Check
  superdesk/items/route.ts     POST  published items from Superdesk
  airtable/webhook/route.ts    POST  change notices from Airtable
```

Each handler first checks that its plugin is enabled, and returns `503` if not. Tests read the route list from the manifests.

4.8 Rules for plugin authors

1. Add only optional fields, inside a group named after the plugin.
2. Read configuration from the context object.
3. Archive, never delete.
4. Keep `health` cheap enough to run on every health check.
5. When disabled, change no behaviour.
6. Wrap any experimental Payload API, such as `definePlugin` or dashboard widgets, in the kernel's own helper, so a change between Payload betas touches one place.

5. Verification plugins

In short: a deployment verifies claims in one place. Each option sends a claim out and brings back the same three fields: a verdict, a date and a rationale. The claim itself never leaves PromiseTracker. Version 1 uses the built-in queue or Meedan Check.

5.1 Built-in review, in plugin-core

A reviewer opens a claim and sees the statement, the source document with the quote highlighted, the actor, and how the claim was found. They set the verdict and write a rationale. A saved view called "Awaiting verification" is the queue. The adapter sends nothing out and fetches nothing.

5.2 plugin-check, version 1

Meedan Check is a fact-checking workspace with a claim type and a report workflow built in. The plugin owns the verdict, verification date and rationale, and adds a read-only `check` group: the item's id in Check, its link, the report state, and the time and result of the last sync.

The API. One GraphQL endpoint, `https://check-api.checkmedia.org/api/graphql`. Every request carries the `X-Check-Token` header with the workspace API key and the `X-Check-Team` header with the workspace id. Both values live in the plugin's settings, encrypted.

How a claim maps to a Check item.

PromiseTracker	Check field	Notes
Statement	`quote`	The exact quote. Check creates an item of type `Claim`
Title and summary	`set_claim_description`	Includes a link back to the claim
Topic, claim type, country	`set_tags`	
Starting status	`set_status`	`undetermined`
Claim identity	The returned item id	Stored as `check.itemId` and used for matching
Item link	The returned `full_url`	Stored as `check.itemUrl`

Sending a claim.

```
mutation createProjectMedia($input: CreateProjectMediaInput!) {
  createProjectMedia(input: $input) {
    project_media { id full_url status }
  }
}
```

```
{
  "input": {
    "media_type": "Claim",
    "quote": "We will build 200 new health centres by 2027",
    "channel": { "main": 1 },
    "set_status": "undetermined",
    "set_tags": ["health", "promise", "ke"],
    "set_claim_description": "Governor: 200 new health centres ..."
  }
}
```

Receiving verdicts. The plugin uses a push and a poll.

- Push: Check posts to `/api/plugins/check/webhook`. The request carries a shared secret in its `Authorization` header, which the handler checks before reading the body. The plugin acts only on report events, which Check labels `report_design`, and acknowledges all others with `200`. A report state of `published` applies the verdict. A state of `paused` withdraws it.
- Poll: every 15 minutes a job searches Check for items with a published report, 100 per page, and reconciles them. A push can be lost during a deploy, and the poll catches it.

```
query PublishedReports($query: String!) {
  search(query: $query) {
    medias { edges { node { id full_url status report_status } } }
  }
}
```

The `query` variable is `{"report_status": ["published"]}`.

From Check status to verdict. Each workspace defines its own statuses, so the plugin's settings map each one to a verdict. At start-up it reads the workspace's statuses and refuses to start if one is unmapped.

Check status, default workspace	Verdict
`verified`	verified
`false`	false
`undetermined`, `in_progress`, `not_applicable`	unverified

Report state	Effect on the claim
None, or not published	In review. No change
`published`	Apply the mapped verdict, copy the report text as the rationale, set the date
`paused`	Set the verdict to unverified, set the claim to hold, notify an editor

When Check fails.

Problem	What the plugin does
An unmapped status at start-up	Refuses to start and names the status
`429`, too many requests	Waits for the `Retry-After` time, then retries

Problem	What the plugin does
A wrong webhook secret	Returns `401` without reading the body
The same webhook twice	Matches a fingerprint of the body and returns the first answer
A webhook for an unknown item	Returns `200` and leaves the item for the next poll
Check unreachable	Pauses calls, queues claims locally, reports a degraded state

The v2 Check client in `src/lib/meedan.ts` and the duplicate detection in `src/app/api/meedan-sync/route.ts` are the starting point for this plugin.

5.3 plugin-superdesk, later version

Superdesk is Sourcefabric's newsroom system. It has no claim type, so the plugin represents a claim as a content item with a dedicated content profile and three custom vocabularies, which the newsroom must set up first.

Topic	Design
Writing	Posts new items to the internal API at `/api/archive`, signed in as a service account with the `archive` privilege, into a named desk and stage. Not to the ingest route, which changes the item's `guid`
Claim fields	Title to `headline`, statement to `body_html`, summary to `abstract`. Claim type, topic and verdict as `subject` entries with their own schemes, because Superdesk can search those. The claim id and link in `extra`
Reading	An HTTP push from a Superdesk subscriber, signed with HMAC-SHA1 in the `x-superdesk-signature` header, plus a 15-minute poll of the Content API filtered on `versioncreated`
States	`published` and `corrected` apply the verdict. `killed`, `recalled`, `takedown` and `unpublished` withdraw it. `spiked` archives the claim
Duplicates	Applies an item only if its version is higher than the stored one
Known traps	The Content API's address prefix differs between installations. Its date parameters take whole days only and default to the last 7 days. It returns an empty list, not an error, if the subscriber matches no product

Appendix A lists the instance details to confirm before this plugin is built.

6. Other plugins

6.1 plugin-ai

Finds candidate claims in documents. It provides an LLM provider adapter for each configured AI vendor, and owns the extraction records and the candidate review screen. A deployment selects one model and stores the vendor's key in encrypted settings. A small test request at start-up and on each health check reports exhausted credit as a degraded state. The provider list and credential checks come from v2's `src/lib/ai/providerCatalog.ts` and `providerRegistry.ts`.

6.2 plugin-tika

Sends files to a self-hosted Apache Tika server and gets back the text. Tika reads PDF, Word, Excel, PowerPoint, OpenDocument, RTF, EPUB and several hundred other formats. Requests time out after two minutes. Tika counts toward the health check only when this plugin is enabled.

6.3 plugin-site

Holds the settings for the default public site, described in section 10.6. It has no port.

6.4 plugin-airtable, later version

Lets a team feed documents and actors in from Airtable. The work queue and processing status stay in PromiseTracker. For most teams, a one-off spreadsheet import is simpler.

6.5 Default adapters for the new ports

Port	Default adapter	Behaviour
SearchProvider	`postgres`	Full-text search with ranking, accent-insensitive and typo-tolerant matching, and place lookups, described in section 16.2
NewsletterProvider	`local`	Stores sign-ups in the database and sends a confirmation email. Nothing is subscribed until confirmed
AnalyticsProvider	`none`	Sends nothing. Searches with no results are still logged locally, section 18
CaptchaProvider	`local`	A proof-of-work check in the browser, a hidden trap field and rate limits. No outside service

7. Data model

In short: a country has languages, areas and offices. Actors hold seats for a term. Actors make claims, which come from documents, carry ratings from a rating scale, and collect dated updates. Editors add topics, themes and articles around them.

```
Country --+-- Language
          +-- Area (nested) ----+
          +-- Office ------------+-- Seat --- Term --- Actor
          |                                            |
          +-- Document --> Extraction --> candidates   | makes
          |                                  | accepted v
          +------------------------------------------ Claim --> Claim updates
                                                       |           ^
              Claim type --> Rating scale --> Ratings --+           | promoted
              Topic, Theme, Article ---------------------+       Submission
```

All records live in PostgreSQL, with UUIDs as ids. Payload keeps a version history and records who created and last changed each record, as described in section 8.5. The tables below leave out those fields and the timestamps. "Localized" means the field holds a value per language. A date marked "partial" stores a precision of year, month or day, because many claims are only known to the month, as in "August 2022".

7.1 Languages and countries

Languages

Field	Notes
`code`	Such as `en`, `fr`, `ar`
`name`	Localized
`direction`	Left to right or right to left
`enabled`	Editors add and enable languages without a code change

Countries

Field	Notes
`code`	Two-letter ISO code in lower case. Unique. Used in web addresses
`name`	Localized
`region`	For grouping countries once there are many
`languages`, `defaultLanguage`	The languages this country publishes in
`rootArea`	The area that stands for the whole country, which national offices such as President belong to
`about`	Localized

Field	Notes
`recordStatus`	ready, hold, archived. Starts as hold

7.2 Areas, offices and seats

Areas are nested with Payload's hierarchy feature, which manages parent links and blocks circular ones.

Field	Notes
`country`	Required
`name`	Localized
`slug`	Unique within the country, and must not match a national office slug or a reserved word, section 11
`type`	Country, region, county, constituency, ward, city or other
`parent`	The area it sits in
`page`	Automatic, always or never. Automatic means the area gets a page when it holds two or more seats
`geo`	Centre point, bounds, and an uploaded GeoJSON outline with its source and licence
`externalCode`	Its id in an outside boundary dataset
`formerSlugs`	For redirects, section 11.2

Offices are each country's list of titles.

Field	Notes
`country`	Required
`name` , `shortName`	Localized, such as "Member of Parliament" and "MP"
`slug`	Used in seat addresses, such as `governor`
`areaType`	The type of area the office answers for. Empty for offices with no area, such as party-list members
`holdersPerSeat`	Usually one. More for multi-member seats
`order`	Display order, highest office first

A **seat** is an area plus an office. It is not stored separately. The system finds a seat's holders by looking for current terms with that area and office.

7.3 Actors and terms

Actors

Field	Notes
`country`	Required
`kind`	person, party or institution

Field	Notes
`name`, `shortName`	
`slug`	Unique within the country, across all kinds
`formerSlugs`	For redirects
`image`, `imageCredit`	
`biography`	Localized
`keyClaim`, `keyClaimNote`	The claim editors pick as most important, and why. Localized note
`links`	Official site and social accounts
`recordStatus`	Starts as hold

People also have an honorific and gender. Institutions have a type, such as ministry or agency.

Terms

Field	Notes
`actor`	Required
`office`, `area`	The seat. `area` is empty for offices with no area
`party`	The actor's party during this term
`electedOn`, `startsOn`, `endsOn`	`endsOn` is empty while in office
`stage`	Computed: candidate before `startsOn`, current during the term, past after `endsOn`
`closingSummary`	Localized. Written when the term ends
`successor`	The actor who took the seat next

An actor whose only terms are at the candidate stage is never public. An actor with no current term and at least one past term shows as an archived record.

7.4 Claim types and rating scales

Claim types

Field	Notes
`name`	Localized, such as "Promise"
`slug`	
`ratingScale`	The scale this type uses
`hasDeadline`	Whether claims of this type usually have a deadline

A new installation has one claim type, Promise, using the Delivery scale. Editors add others, such as policy pledge, budget commitment or public statement, each with a suitable scale.

Rating scales contain their tiers and ratings.

Field	Notes
`name`	Localized, such as "Delivery"
`tiers`	Each with a label, colour and order
`ratings`	Each with the fields below
`bestValue`, `worstValue`	For structured data, section 10.7

Rating field	Notes
`slug`	Stable, such as `in-progress`
`label`	Localized. Editable at any time
`definition`	Localized. Editable for wording only. Every change is kept in the version history
`tier`	The tier it belongs to
`colour`, `textColour`	The admin checks contrast against WCAG AA
`value`	A number on the scale, for structured data
`needsDeadline`	A claim with this rating must have a deadline
`isStarting`	The rating every new claim begins with
`retired`	A retired rating cannot be given to new claims. A rating in use cannot be deleted

The default Delivery scale:

Tier	Rating	Needs a deadline
Promise kept	Completed	No
Promise kept	In Progress	No
Uncertain	Unstarted, the starting rating	No
Uncertain	Inconclusive	No
Promise not kept	Behind Schedule	Yes
Promise not kept	Stalled	Yes

If the line between two ratings has to move, the affected claims are re-rated through normal claim updates. Retiring a rating lists the claims that carry it so editors can re-rate them.

7.5 Claims

Field	Notes
`country`	Required
`type`	Claim type. Starts as Promise
`title`	Localized. The headline

Field	Notes
`slug`	Unique within the country
`statement`	The exact words. Never paraphrased, never translated
`summary`	Localized. Editorial context
`actor`	Required. Who made the claim
`coActors`	Such as the party alongside the candidate
`term`	The term it belongs to, if any
`area`	Where it applies, if narrower than the actor's seat
`topic`	Required to publish
`tags`	Payload tags, free-form
`source` , `sourceLocator`	The source document and page. A source document or at least one piece of evidence is required to publish
`sourceUrl`	Shown as a link. Never fetched by the server
`saidAt` , `saidOn`	Where and when it was said. `saidOn` is a partial date
`deadline` , `deadlineText`	A partial date, and the written form such as "End of term"
`currentRating` , `ratedOn`	Read-only. From the latest published update
`lastReviewedAt`	Read-only. Set by an update or by confirming the rating, section 8.4
`updateCount`	Read-only
`verification`	Verdict (unverified, verified, false, misleading), date, who verified it, rationale, and any published report
`extraction` , `confidence` , `enteredBy`	How the claim was found: AI, by hand or by import
`image` , `imageCredit`	
`recordStatus`	Starts as hold

7.6 Claim updates and evidence

Claim updates

Field	Notes
`claim`	Required
`date`	Partial date. When the event happened, which may be long before it was recorded
`rating`	The rating as of that date
`kind`	first rating, change, or correction of our own earlier mistake
`title` , `narrative`	Localized. What happened and how we know
`evidence`	At least one item when the rating changes

Field	Notes
`attribution`	Credit for a citizen, with consent
`submission`	Set when a moderator promoted a submission
`recordStatus`	Starts as hold

Evidence

Field	Notes
`kind`	Document, dataset, social post or article
`title`, `publisher`, `date`	
`link` or `file`	Links are shown, never fetched
`finding`	Localized. One line saying what the evidence shows
`thumbnail`	

When a published update is added, changed or removed, the system sets the claim's `currentRating`, `ratedOn`, `lastReviewedAt` and `updateCount` from its published updates.

7.7 Editorial content

Record type	Fields
Topics	Name, slug, description, colour, and one level of nesting through Payload's hierarchy. Shared across countries
Themes	Title, introduction, image, a hand-picked list of claims, and the countries it covers
Articles	Title, standfirst, body, image, date, what it is about (a claim, an actor or a country), byline, and a switch to hide the byline. No byline is a normal state
Pages	Editor-made pages such as FAQ and resources, nested with hierarchy. A page can instead be a marked link to a sister site
Partners	Name, logo, link, countries
Sister tools	Name, description, link, for "Other tools from PesaCheck"

7.8 Documents and extractions

Documents hold source material: country, title, actors, type, language, publisher, date, the stored file, the source link (shown only), the connector that brought it in, the detected file type, the extracted text, and a processing status (section 9.4) with its message and retry count. Documents are not public. Claim pages link to them as evidence.

Extractions, owned by plugin-ai, record one AI run over one document: the model, the prompt version, the status, tokens used, and the candidates. Each candidate has a summary, exact quotes, a suggested topic, a confidence, and a decision of pending, accepted or rejected with the reviewer and reason. Running AI again creates a new extraction and keeps the old ones.

7.9 Submissions, sign-ups and logs

Record type	Fields
Submissions	Claim, kind (new evidence, correction, or "this has changed"), text, location, files, evidence link (never fetched), optional name and contact, consent to be credited, status (new, accepted, rejected, spam), moderator and note, a scrambled form of the sender's network address
Newsletter sign-ups	Email, language, confirmed or not, confirmation date. Used by the `local` newsletter adapter only
Rating reviews	Claim, reviewer, date, and whether the rating was confirmed or changed
Search log	Query, language, country, number of results. Nothing about the person

7.10 Users, media and settings

Users have a name, roles (section 13.2) and a country list that version 1 does not enforce.

Media uses Payload 4 uploads: the original file is kept, crops and focal points are stored, image sizes are made on request, and every file has a credit field. Files are matched by checksum, so the same file is stored once.

Settings, a global, holds feature switches, the counts-not-percentages threshold (20 by default), and each plugin's credentials. Only super-admins can read it, and every credential is encrypted. Environment variables choose which plugins run. The database holds what they need.

7.11 Indexes and search columns

Record type	Indexes
Areas	Country and slug, unique. Parent
Actors	Country and slug, unique. Kind
Terms	Actor. Office and area. End date
Claims	Country and slug, unique. Actor. Topic. Current rating. Deadline. Last reviewed
Claim updates	Claim and date, newest first
Documents	Processing status and last attempt
Submissions	Status. Claim

Claims, actors, areas and topics each have a full-text search column per language, and actor and area names have a trigram index for misspellings. Section 16.2 covers the Postgres extensions.

8. Publishing rules

In short: an editor sets each record to ready, hold or archived. The system works out what is live, for each language. A claim is live only when it is ready, everything above it is ready, it is verified, it has a source and a topic, and it has the fields it needs in that language.

8.1 Two fields

`recordStatus` is what the editor wants: ready, hold or archived. It is one value per record.

The published status is what the system computes, for each language. A record is published in a language when:

- its own record status is ready
- every record above it is ready
- its required localized fields exist in that language
- for a claim, its verdict is verified, it has a topic, it has a source document or evidence, and it has a deadline if its rating needs one

Record	Records above it
Country	None
Area	Its country
Actor	Its country. An actor also needs a current or past term
Document	Its country
Claim	Its country, its actor, and its source document if it has one
Claim update	Its claim, and everything above the claim
Article	What it is about

A claim's source document sits above it because the document is the evidence. Setting a fake manifesto to hold removes every claim drawn from it.

Two fields are needed because one switch cannot say "this claim is fine, but its source is under review". With two, a claim an editor held on its own stays held when its country is released, and a claim hidden only because of its country comes back.

Payload 4 publishes the language being edited. "Publish all languages" is a separate action.

8.2 The cascade

When an editor changes a record's status:

1. The system recomputes that record's published status in every language before the save ends.
2. A background job recomputes every record under it, in batches, inside database transactions.
3. When the job ends, the site's cached pages for those records are refreshed.

The public site asks one question of every list: is this record published in this language? It never checks parents.

8.3 A language that is missing

If a visitor asks for a record in a language it is not published in, the API answers with a "not available in this language" response that names the languages it is published in. The site shows a short notice with a link. English text never appears on a French page.

8.4 Reviewing a rating

Every claim shows when its rating was last reviewed. An editor who checks a claim and finds nothing new uses "Confirm rating", which records a rating review and moves `lastReviewedAt` without adding a public update. A change of rating goes through a normal claim update, which also moves the date. A dashboard widget lists the claims not reviewed for the longest time.

8.5 History and authorship

Payload keeps a version history of every record type except users, and stamps each record with who created and last changed it. This is internal history. The public history of a claim is its claim updates.

Version history and the created-by and changed-by fields are readable by signed-in users only. Showing them publicly would reveal which staff member edited a claim, which would undo the protection of an optional byline.

8.6 Deletion

No sync, import or plugin deletes a record. A deletion upstream archives the record here. An editor can delete a record by hand, and it goes to Payload's trash first, from which it can be restored.

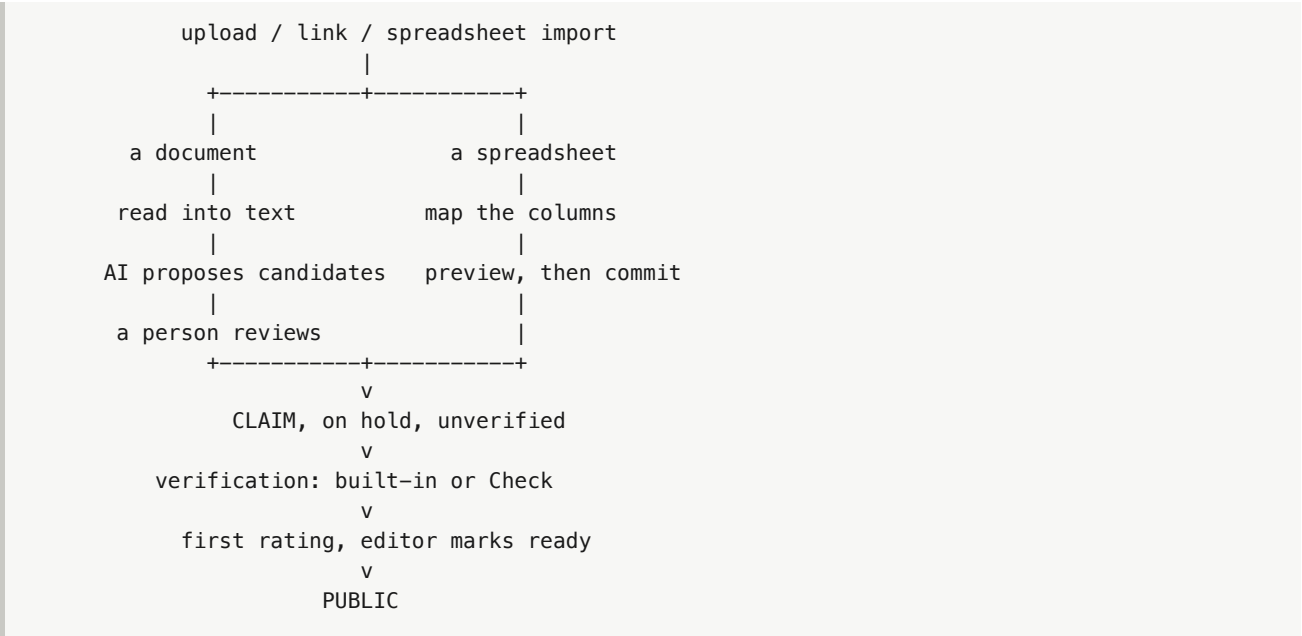
8.7 Audit trail

Event	Recorded in
Any change to a record	Version history, with who made it
A change of record status	Version history and a log entry with the old and new value
A verdict	The claim's verification fields
A rating confirmed	Rating reviews
A candidate accepted or rejected	The candidate
A submission moderated	The submission
A super-admin overriding a plugin's field	A warning in the log with the old and new value

9. The pipeline

In short: content enters through a connector. Files are read into text, AI proposes candidates, and a reviewer accepts or rejects them. Spreadsheets skip those steps and become records directly. Either way the result is a claim, which is verified, rated and published.

9.1 The two paths



9.2 Connectors

Connector	Plugin	How it works
`upload`	core	An editor uploads a file
`url`	core	An editor pastes a link and the server fetches it through the safeguards in section 13.4
`airtable`	airtable, later	A scheduled job reads new rows

The link connector is the only place where the server fetches an address a user typed. It stores the file, so it never fetches the address again.

9.3 Reading files

The system checks a file's first bytes, then the declared type, then the extension. If they disagree, the document fails with a message saying why.

Extractor	Plugin	Reads
`plaintext`	core	Plain text, Markdown, and HTML with the page furniture removed
`tika`	tika	PDF, Word, Excel, PowerPoint, OpenDocument, RTF, EPUB and more

An extractor returns text in a standard Unicode form and keeps every character. Version 2 strips accented characters, which damages every French document.

A spreadsheet can be a document to read or a list of records. The editor chooses at upload. CSV defaults to records, Excel to a document.

9.4 Processing status

Each document has one status, which is also the work queue:

```
pending > fetching > fetched > extracting > extracted > analysing > analysed > completed
Spreadsheet path:  pending > imported > completed
From any state:    failed, or needs-review
```

Every job writes the status before and after its work, with any error message, so the person who uploaded a file can see what happened.

9.5 AI extraction and review

1. The text is split into overlapping sections. Each goes to the selected model, which returns candidates with a topic, a summary and exact quotes. The instructions ask for specific commitments, forbid merging two into one, and require quotes in the document's own language.
2. Duplicates are removed, then every candidate whose quote does not appear word for word in the document is discarded. This stops the AI from inventing claims.
3. A second pass splits compound commitments and writes titles. The quote check runs again.
4. The result is one extraction whose candidates all start as pending.

A reviewer sees each candidate with its quotes highlighted in the source, and accepts, edits and accepts, or rejects it with a reason. Accepting creates a claim on hold and unverified.

9.6 Spreadsheet import

1. Inspect: read the headings and the first rows.
2. Map: match columns to fields. The mapping is saved for reuse.
3. Preview: run without saving, and report rows to create, update and skip, with errors by row.
4. Commit.

An import can create areas, offices, actors, terms, claims, claim updates and documents, which is also how a country's lists of offices and areas are first loaded. Each record type has a natural key, such as country and slug, so importing the same file twice updates records instead of duplicating them.

9.7 Handling failures

Error	Retries	Then
Network timeout	3, with growing waits	Next item
Too many requests	After the wait the service asks for	Back in the queue

Error	Retries	Then
Service unavailable	3	The next scheduled run retries
Invalid data	None	Mark failed with the reason
Sign-in or permission error	None	Fail and alert

Waits start at one second, double each time, and stop at 30 seconds. File fetches time out after 5 minutes, text extraction and each AI call after 2 minutes, and a document's whole AI stage after 20 minutes.

Payload 4 handles crashed workers. Each running job holds a lease, and if its worker dies, another worker picks the job up when the lease runs out. Concurrency keys stop two workers running the same document, and limit calls to each AI provider.

After 5 failures in a row, the system stops calling an outside service for 60 seconds, then tries 3 test requests before resuming. Developers call this a circuit breaker.

9.8 Background jobs

Job	Plugin	Runs when
`fetchDocument`	core	A document is created with a link
`extractText`	core	A document reaches fetched
`extractCandidates`	ai	A document reaches extracted
`runImport`	core	An editor commits an import
`submitForVerification`	check	A claim is created
`pullVerdicts`	check	Every 15 minutes
`recomputeClaim`	core	A claim update changes
`cascadeStatus`	core	A record status changes
`refreshCounts`	core	After a cascade or a rating change
`cleanupJobs`	core	Hourly

10. The UI layer

In short: the public site is a set of pages built from sections. Each section is a component that receives data through props and decides whether it has anything to show. The site gets its data only from the SDK, so any section, page or the whole site can be replaced.

10.1 Packages

Package	Holds	May import
`@promisetracker/contract`	A schema for every API response, written with Zod. TypeScript types and the OpenAPI document are generated from it	Nothing from the platform
`@promisetracker/sdk`	A typed client, and page view builders such as `getClaimView`	The contract
`@promisetracker/ui`	The redesign's components: atoms, cards and sections. Props only	The contract's types
`apps/site`	Routes, page templates, share images	The SDK and the UI package. Never Payload

A lint rule enforces the last column.

The SDK has two transports. The local transport calls Payload in the same process with no network hop, and is used while the site and the platform run in one deployment. The HTTP transport calls the API, and lets the site run anywhere. Switching between them is a one-line change.

10.2 Pages and sections

A page template is an ordered list of sections. Each section is registered with an id, a component, and a rule for when it shows. A deployment can swap a section for its own component, reorder sections, or replace a page.

Page	Sections
Home	What this is. A real claim with its rating and sources. In focus, a theme. How a rating is decided. What we have tracked. Choose a country. Who we work with. Other tools from PesaCheck
Country	In focus. Country snapshot. Who is delivering. Officials. Parties. Past administrations
Actor	Who this is. Key claim. Record so far. Claim list. Latest changes. Latest analysis. Closing summary for past actors
Claim	The claim, with who, where, when and by when. Rating and last reviewed date. How the rating changed. Evidence. Analysis. Submit an update
Area	The office holders who answer for it, with combined figures and a note that the figure adds several people's claims
Topic, theme	Claims on the subject, with figures by country

Page	Sections
Search	Places with their representatives, then actors, claims and topics
Others	Officials list, rating history, submit an update, methodology, editor pages

Every page also has the header (name, a one-line description, search, and the "verified by" mark), a short explanation of the site collapsed by default, and the footer.

10.3 When a section shows

A section with nothing to show is left out. It is never shown empty, greyed out, or as a zero. The API returns the counts each rule needs, and editors can force any section on or off through plugin-site.

Section	Shows when	Otherwise
Who is delivering	Two or more current office holders	Left out. The country page leads with the one office holder
Officials	Two or more current office holders	Left out
Choose a country	Two or more countries	A single link to the one country
In focus, home	A theme spanning two or more countries	A theme within the one country, or left out
Past administrations	At least one ended term	Left out
Closing summary	The term has ended	Left out
Latest analysis	At least one published article	Left out

10.4 Counts, not percentages

Below the threshold in Settings, 20 claims by default, figures are shown as counts, "1 of 3 completed", rather than percentages. The API returns which form to use with every set of figures, so pages, share images and embeds agree.

10.5 Languages, accessibility and weight

- Interface text comes from a dictionary in plugin-site. Editors can add a language and translate the interface without a code change.
- Layouts use logical CSS properties, so right-to-left languages such as Arabic work.
- Ratings are never shown by colour alone. Contrast meets WCAG AA, including every rating colour.
- The timeline is a real ordered list, and every control works with a keyboard.
- Pages are built for phones on slow, paid data first. Images must earn their place.

10.6 Settings in plugin-site

Setting	Purpose
Homepage picks	The featured claim, the In focus theme, the featured area
Country picks	The In focus actors and claims
Section overrides	Force a section on or off
Explainer	The one-line description and the longer collapsed explanation
Navigation and footer	Links, partners, sister tools
Interface strings	Per language
Theme	Colour, type and spacing tokens

10.7 Share images and structured data

Each page type has its own share image, made from its real content: the home figures, a country's headline figure, an actor's record and one claim, a claim and its rating. They follow the counts rule.

Every verified claim page carries ClaimReview data from schema.org, which search engines read to show fact-checks. The rating's `value` and its scale's best and worst values become the rating number. Actor pages carry Person or Organization data.

11. Web addresses

11.1 Pages

The language comes first, then the country. Addresses are lower case with no trailing slash. `e` marks an actor and `p` a claim, for every kind and type, which keeps shared links short.

Page	Address	Example
Home	`/{lang}`	`/en`
Country	`/{lang}/{cc}`	`/en/ke`
Officials list	`/{lang}/{cc}/officials`	`/en/ke/officials?office=governor&sort=kept`
Actor	`/{lang}/{cc}/e/{actor}`	`/en/ke/e/achieng-wekesa`
Claim	`/{lang}/{cc}/p/{claim}`	`/en/ke/p/kibra-housing`
Rating history	`/{lang}/{cc}/p/{claim}/history`	`/en/ke/p/kibra-housing/history`
Submit an update	`/{lang}/{cc}/p/{claim}/update`	`/en/ke/p/kibra-housing/update?kind=correction`
Area	`/{lang}/{cc}/{area}`	`/en/ke/nairobi-county`
Seat	`/{lang}/{cc}/{area}/{office}`	`/en/ke/nairobi-county/governor`
National seat	`/{lang}/{cc}/{office}`	`/en/ke/president`
Article	`/{lang}/{cc}/a/{article}`	`/en/ke/a/why-the-housing-money-stopped`
Topic	`/{lang}/topics/{topic}`	`/en/topics/health?country=ke`
Theme	`/{lang}/themes/{theme}`	`/en/themes/water-and-sanitation`
Search	`/{lang}/search`	`/en/search?q=nairobi`
Methodology	`/{lang}/methodology`	`/en/methodology`
Editor pages	`/{lang}/{page}`, nested	`/en/about`, `/en/resources/guides`

11.2 How addresses behave

- **An actor's address never changes.** A past office holder keeps it, and it shows the archived record.
- **A seat address follows the seat.** It sends a temporary redirect to whoever holds the seat today. A seat with several holders lists them.
- **An area address with one seat redirects** to that seat's holder. An area with several seats shows its page if it has one, and otherwise lists its seats.
- **One namespace per country.** The segment after `/{cc}/` can be an area, a national office or a reserved word, so these must not collide. The reserved words are `officials`, `e`, `p`, `a`, `search` and `feed.xml`.
- **A renamed slug keeps working.** The old slug sends a permanent redirect to the new one.

- `/` **chooses a language,** the visitor's preferred enabled language, otherwise English.
- **The language switch keeps the page.** Slugs are not translated, so only the language changes.
- **A missing translation shows a notice,** section 8.3.

11.3 Machine addresses

Purpose	Address
Share images	`opengraph-image` under each page
Country feed	`/{lang}/{cc}/feed.xml`
Sitemaps	`/sitemap.xml`, one child per language and country
Robots	`/robots.txt`
API	`/api/v1/...`
Health	`/api/health`, `/api/health/ready`
Plugin routes	`/api/plugins/{plugin}/...`
Admin	`/admin`
Embeds, version 2	`/embed/{lang}/{cc}/p/{claim}`, `/embed/{lang}/{cc}/e/{actor}`
oEmbed, version 2	`/api/oembed?url=`

Payload's own REST and GraphQL routes stay available to the admin and signed-in users. The site never calls them.

12. The API

In short: a read-only API, versioned, that returns published content only, in one language at a time. The public site uses it from day one. Outside users get access in version 2.

12.1 Endpoints

Endpoint	Returns
`GET /api/v1/site`	Languages, rating scales, thresholds, homepage picks, overall figures
`GET /api/v1/countries`	Countries with headline figures
`GET /api/v1/countries/{cc}`	One country, its figures, picks, parties and section counts
`GET /api/v1/countries/{cc}/offices`	The country's offices
`GET /api/v1/countries/{cc}/areas?parent=`	Areas, one level at a time
`GET /api/v1/countries/{cc}/areas/{area}`	An area, its seats and holders, combined figures
`GET /api/v1/countries/{cc}/seats/{area}/{office}`	The current holders of a seat
`GET /api/v1/countries/{cc}/actors`	Actors, filtered by kind, stage, office, area or party, and sorted
`GET /api/v1/countries/{cc}/actors/{slug}`	An actor, their terms, record, key claim, recent changes, closing summary
`GET /api/v1/countries/{cc}/claims`	Claims, filtered by actor, type, topic, rating, tier or deadline, and sorted
`GET /api/v1/countries/{cc}/claims/{slug}`	A claim, its rating, evidence, articles and citation text
`GET /api/v1/countries/{cc}/claims/{slug}/history`	The rating history
`GET /api/v1/topics` , `/topics/{slug}`	Topics, with figures by country
`GET /api/v1/themes/{slug}`	A theme and its claims
`GET /api/v1/articles?about=`	Articles about a claim, actor or country
`GET /api/v1/search?q=&country=`	Areas with their representatives, then actors, claims and topics
`GET /api/v1/redirects?path=`	Where a former slug or a seat address leads
`POST /api/v1/submissions`	A citizen submission
`POST /api/v1/subscriptions`	A newsletter sign-up

12.2 Rules every endpoint follows

- A `locale` parameter is required. A record not published in that language returns the "not available" response in section 8.3.
- Lists use cursor pagination and return a total.
- Rating figures always have the same shape: counts by rating, counts by tier, the total, and whether to show counts or percentages.
- Addresses use country and slug. Records carry UUIDs.
- Responses carry `Cache-Control` and `ETag`. Publishing refreshes the cache.
- Read routes skip authentication, which Payload 4 allows per endpoint.
- Created-by, changed-by and version fields never appear.
- The two POST routes need a captcha pass and are rate limited.
- The OpenAPI description is at `/api/v1/openapi.json`.

13. Security

In short: the public site needs no sign-in and shows published data only. Everything else is behind roles. The server does not fetch addresses users type, files are checked before they are stored, credentials are encrypted, and staff identities stay private.

13.1 Who can reach what

Area	Protection
Public site and API read routes	None
Public forms	Captcha and rate limits
Admin	Signed-in session, with Payload 4 refresh tokens
Plugin routes	A shared secret or signature per route
AI assistant access through Payload's MCP plugin	Signed in through its built-in OAuth server, limited to the user's own permissions. Never public

Payload 4 enforces access rules in server-side calls by default. Code that must bypass them, such as jobs and seeding, says so explicitly.

13.2 Roles

A user can hold more than one role.

Role	Purpose
super-admin	Full access. Users, settings, credentials, rating scales, offices and areas
editor	Decides what is public. Rates claims, owns claim updates, sets picks
contributor	Adds actors, documents and draft claims. Cannot publish
reviewer	Reviews AI candidates and sets verdicts. Loses the verdict when a verification plugin is on
moderator	Handles submissions and can turn one into a claim update
viewer	Read-only

Action	super-admin	editor	contributor	reviewer	moderator
Manage countries, languages, offices, areas	Yes				
Manage rating scales and claim types	Yes				
Create or edit actors, terms, documents	Yes	Yes	Yes		
Create claims by hand	Yes	Yes	Yes		
Accept or reject AI candidates	Yes	Yes		Yes	

Action	super-admin	editor	contributor	reviewer	moderator
Set a verdict	Yes	Yes		Yes	
Rate a claim, add or confirm updates	Yes	Yes			From a submission
Set record status	Any	Any	Hold only		
Moderate submissions	Yes	Yes			Yes
Edit picks, themes, articles, pages	Yes	Yes			
Delete a record, to the trash	Yes	Yes			
Override a plugin's field	Yes, logged				

Version 1 does not limit editing by country. Every record already carries its country, so adding the limit in version 2 is a filter in the access rules. It must be in place before an outside newsroom gets an account.

13.3 Checking input

All outside data is checked against a schema before use: webhooks, API responses, spreadsheet rows and form submissions. A webhook handler checks the signature or secret before reading the body, then checks for a repeated delivery, then the shape, and only then writes.

13.4 Fetching outside addresses

A server that fetches any address a user types can be tricked into reaching private systems. This attack is called server-side request forgery, or SSRF.

By default the server fetches nothing a user typed. Source, evidence and submission links are stored and shown, never fetched. The link connector is the one exception, and it applies these safeguards, all from v2's `src/utils/ssrf.ts` and `src/utils/files.ts`:

- HTTPS only.
- The address is looked up first, and any private, local, reserved or cloud metadata address is refused.
- Redirects are followed one at a time, each checked the same way, up to a limit.
- Time limits per hop and in total, and a size limit checked from the declared size and again while downloading.
- The file type must match the file's first bytes. SVG files are refused.

13.5 Uploads and content

- The extension, the declared type and the first bytes must agree.
- Each kind of file has a size limit, and each client a rate limit and a daily volume limit.
- Extracted text and outside HTML are reduced to a short list of safe tags before storage.
- No visitor's network address is stored. Submissions keep a scrambled form of it, enough to spot repeated abuse and not enough to identify anyone.

13.6 Staff privacy

Version history and the created-by and changed-by fields are readable by signed-in users only, and never appear in the API. Article bylines are optional, and no byline is a normal state. In some countries naming the person who wrote a piece puts them at risk.

13.7 Credentials

Credential	Where it lives	Replaced
Database and application secrets	Environment	Yearly
File storage keys	Environment	Yearly
Plugin API keys and tokens	Settings, encrypted	When a staff member with access leaves
Webhook secrets	Environment	Every 6 months

The encrypted settings field from v2, `src/fields/encryptedSecret.ts`, encrypts on save, shows a mask in the admin, and keeps the stored value when a form is saved without retyping it. Logs never contain keys, tokens, passwords, email addresses or network addresses.

13.8 Browser security

No other site may place PromiseTracker's pages in a frame. Embed routes, in version 2, are the one exception, and they carry no cookies, sign-in or state.

13.9 Personal data

Kenya's Data Protection Act and its equivalents in the other launch countries apply to submitter details, newsletter sign-ups and analytics.

- A privacy notice explains what is collected and why.
- Newsletter sign-ups are confirmed by email before use.
- Analytics, when enabled, sets no cookies and stores nothing that identifies a person.
- Submitter contact details are deleted on request and never published without consent.

Data	Kept for
Claims, updates, actors, documents	Indefinitely. Archived, never removed automatically
Raw AI responses	90 days. Candidates and decisions are kept
Rejected and spam submissions	180 days
Search log	12 months
Failed background jobs	7 days

14. Errors and logging

Every error has a stable code, the ids of the records involved, and a flag saying whether a retry can help. Retry logic reads the flag.

Error type	Raised when	Retried
ConfigurationError	A setting is missing or wrong at start-up	No. The system does not start
PortResolutionError	A port has no adapter, or more than one	No. The system does not start
ExternalServiceError	An outside service fails or times out	Depends on the response
ExtractionError	Reading a file or running AI fails	Yes, within the retry budget
ValidationError	Data has the wrong shape, or a claim fails a publishing check	No
OwnershipError	Something writes a field a plugin owns	No

Logs are structured, and carry the level, message, time, source, and the ids needed to follow one document through the pipeline. Successes are logged as well as failures. Entries go to the console and to Sentry.

A problem an editor can fix shows in the admin on the record: a file that cannot be read, a claim that fails a publishing check, a spreadsheet row with an unknown actor. A problem only a developer can fix goes to Sentry.

Event	Email
A record is archived or restored	Yes
A country is set to hold	Yes
A document fails after all retries	Yes, to the uploader
New submissions	A daily digest to moderators

15. Testing

Kind	Share	Covers
Unit	About 80%	The publishing rule, slugs, the quote check, column mapping, partial dates, the counts rule, section visibility rules
Integration	About 15%	Jobs with outside services simulated, routes, access rules, the cascade, plugin loading, the API contract
End to end	About 5%	A document from upload to public page. A submission from form to timeline. A seat redirect after a term ends

Coverage targets are 70% overall and 85% for the pipeline, publishing rules and plugin loading.

Rules specific to this design:

1. Each plugin is tested switched on and off. Switched off, it changes no behaviour.
2. The core is tested alone, with no credentials. If those tests need an API key, rule 1 in section 1.5 is broken and the build fails.
3. The API responses are checked against the contract package.
4. The site builds without importing Payload.
5. The v2 security tests carry over and must keep passing.

16. Deployment and operations

16.1 The Docker image

One Docker image holds the whole application.

Process	Command	Runs
Web	The default	The public site, the admin and the API
Worker	The same image with the worker command	Background jobs

Postgres and file storage are outside the image. Tika runs in its own container when its plugin is enabled. A Compose file brings up the image, a worker, Postgres, a local S3-compatible store and Tika, so anyone can run the whole stack on one machine.

New versions roll out without downtime. An instance receives traffic only after its readiness check passes. Database migrations run before the new version serves traffic.

16.2 PostgreSQL

- Every schema change goes through a migration, created with `payload migrate:create` and committed. CI fails if the configuration and the committed migrations disagree.
- Two extensions are required, and start-up checks that both are installed:
  - `unaccent`, so "Senegal" finds "Sénégal". Payload ships an operator for it.
  - `pg_trgm`, so a misspelt name still matches.
- Full-text search uses a column per language with a GIN index.
- "Who represents this place" walks the area tree up and down in one recursive query.
- Rating counts by actor, area, topic and country are grouped queries, cached and refreshed after a cascade or a rating change.
- The cascade runs inside transactions.
- Read replicas can be added for heavy traffic. The Postgres adapter supports them.
- PostGIS is optional. It would add "find my area from my location" later.

CONFIRM that the chosen managed Postgres provider offers both required extensions.

16.3 Health checks

`/api/health` reports the enabled plugins, the adapter for each port, and the state of each service. `/api/health/ready` checks only the database and file storage. If Tika, the AI provider or Check is down, the pipeline slows or pauses and the public site keeps serving.

```
{
  "status": "healthy",
  "plugins": { "enabled": ["core", "site", "tika", "ai", "check"] },
  "services": {
    "database": { "healthy": true, "latencyMs": 4 },
    "storage":  { "healthy": true, "latencyMs": 22 },
    "tika":     { "healthy": true, "latencyMs": 31 },
    "llm":      { "healthy": true, "latencyMs": 180 },
    "check":    { "healthy": true, "latencyMs": 95 }
  }
}
```

16.4 Alerts and dashboards

Condition	Goes to
Pipeline success below 90% for 15 minutes	Team chat
A service unhealthy for more than 5 minutes	On-call
Error rate above 5% for 10 minutes	Team chat
A rise in rejected webhooks	Team chat
AI account out of credit	Team chat

Admin dashboard widgets, from Payload 4, show editors the review queue, claims waiting for verification, claims not reviewed for longest, waiting submissions, claims that fail a publishing check, and searches that found nothing.

16.5 Backups

The managed database keeps point-in-time recovery, so at most a few minutes of data can be lost, plus a daily full backup kept for 30 days. File storage keeps earlier versions of each file in a second region. The target is recovery within 4 hours. The team tests a restore every quarter.

16.6 A new installation

Migrations create the default Promise claim type and the Delivery rating scale. Then, in order:

Order	What	Minimum
1	Languages	English and French
2	A country	One
3	Its areas and offices, usually by spreadsheet import	The levels you will track
4	Topics	8 to 15
5	Users	One super-admin and one editor
6	plugin-site settings	Enough to show a page
7	Actors and their terms	The people you will track

17. Performance

Measure	Target
A cached page at the server	Under 100 ms
An API response	Under 300 ms for 95% of requests
Main content visible on a 4G phone	Under 2.5 seconds
JavaScript on a claim page	Under 150 KB compressed
A document through the whole pipeline	Under 15 minutes for 95% of documents

Growth	Version 1	Version 2	Later
Countries	4	20	50 or more
Actors	200	2,000	10,000
Claims	2,000	25,000	150,000
Monthly visitors	20,000	200,000	1,000,000

Pages for published records are built ahead of time and cached with tags, using Next.js 16 cache components. A change refreshes only the pages tagged with the changed records.

Tag	Refreshed when
`country:{code}`	The country or its figures change
`area:{id}`	The area or its office holders change
`actor:{id}`	The actor, or a claim naming it, changes
`claim:{id}`	The claim or one of its updates changes
`ratings`	A rating scale changes
`site`	Site settings or picks change

18. Measures

These come from the product definition. Each has a number that would make us change something.

Measure	Source	Why
Time since each claim was last reviewed	Rating reviews and updates	If half the record has not been looked at in a year, the site is stale. Reported to the team monthly
Searches that found nothing	The search log	Tests whether location search works, and lists missing places
Submissions received, and how many changed the record	Submissions	Few coming in means the form is invisible. Many with no changes means noise or a backlog
First visits by page type	The analytics port	Tests whether most visitors arrive from shared links
Links from news sites	Checked by hand each quarter	Whether journalists use it
Before launch: every published claim has a source and a topic, and every claim with a "needs a deadline" rating has one	Publishing checks	These are enforced, so the check is that nothing is waiting on them

19. Development and delivery

19.1 Repository

A new monorepo, managed with pnpm workspaces.

```
apps/
  site/               public site: routes, page templates, share images
  cms/                Payload config and admin, mounted in the same Next.js app at launch
packages/
  kernel/             ports, plugin loader, start-up checks
  contract/           API schemas
  sdk/                typed client and page view builders
  ui/                 components from the redesign
plugins/
  core/  site/  check/  ai/  tika/  superdesk/  airtable/
docker/
  Dockerfile  compose.yaml
```

The redesign repository's components move into `packages/ui`, and its routes into `apps/site`. Named modules from v2 carry over, listed in Appendix B.

19.2 Working on the code

Task	Command
Install	`pnpm i`
Run locally	`pnpm dev`
Start Postgres, storage and Tika	`docker compose up -d`
Create a migration	`pnpm payload migrate:create`
Lint and format	`pnpm lint`
Tests	`pnpm test`
Build the image	`docker build -f docker/Dockerfile .`

`main` is the production branch. Work happens on branches and merges by pull request, with Conventional Commits messages. CI runs the linter, the migration check and the tests against a Postgres service, with only the core plugins enabled and no credentials.

Payload 4 is pinned to an exact beta. Each beta upgrade gets its own branch, runs Payload's codemods, and passes the full test suite before merging.

19.3 Versions

Version	What is built	Done when
1	Everything v2 does, on the new platform, with the new design. Kernel and plugins. The data model. Publishing rules. Upload, link and spreadsheet import. plugin-tika, plugin-ai and the candidate review. Verification in the built-in queue and plugin-check. Ratings, history and rating reviews. Submissions and the newsletter. The public site with search, share images and structured data. The API, used by the site. The Docker image	A Kenyan editor imports the offices and areas, adds an actor, extracts claims from a PDF, verifies one in Check, rates it, publishes it in English and French, and finds it by searching for the county, with no Airtable anywhere
2	Editing rights per country. Embeds and oEmbed. The API opened to outside users. Maps	An editor limited to one country cannot edit another
Later	plugin-superdesk. plugin-airtable. Automatic share image refresh. Petitions	Each one is planned when it is picked up

20. Decisions

20.1 Register

#	Decision
D1	Verification happens in one place per deployment: the built-in queue, Meedan Check or Superdesk
D2	The system tracks claims of every type. Promise is the default type
D3	Plugins and ports. Nine ports, plus Payload's own adapters for the database, storage and email
D4	The core runs on Postgres and file storage alone
D5	A plugin's fields are read-only in the admin to everyone but the plugin
D6	The Check plugin is fully specified and built in version 1
D7	Claims have a public, dated history, with corrections marked
D8	AI extraction stays, simplified, with the quote check
D9	Citizen input is native and moderated
D10	No multi-tenancy. Country is part of the address and a field
D11	Caching, paging, search and structured data are duties of the UI layer, which the API makes possible
D12	v3 is a new codebase in a new monorepo, reusing named v2 modules
D13	The core is a plugin
D14	Actors are people, parties and institutions
D15	Replaced by D36
D16	No editing rights per country in version 1. First in version 2
D17	Geodata lives on areas. Maps come in version 2
D18	Ingestion splits into connectors, extractors and importers
D19	PostgreSQL
D20	Payload 4 with all its new features, pinned until 4.0
D21	The PesaCheck product definition is a source of requirements alongside these decisions
D22	Editors define ratings in rating scales. The six ratings in three tiers are the default
D23	Places are areas in a nested list, not actors
D24	Page structure belongs to the UI layer
D25	The public site reads data only through a typed SDK and contract
D26	The site and the platform run as one deployment at launch
D27	The API is used by the site first and opened to outside users in version 2
D28	No live reader counter until real analytics exist
D29	The database schema always includes every first-party plugin

#	Decision
D30	A missing translation shows a notice and a link, never the other language's text
D31	Embeds come in version 2
D32	A self-hosted captcha by default
D33	MIT licence
D34	Shipped as one Docker image
D35	Version 1 matches what v2 does today, with verification in the built-in queue or Check
D36	Short address letters: `/e/` for actors, `/p/` for claims

20.2 Why

Why Payload? It is the only option we compared that is MIT licensed, runs on Postgres in TypeScript, includes drafts, version history, per-language publishing, row-level access rules, a job queue and a plugin system for free, and runs inside the same Next.js app as the site. Strapi keeps version history and audit logs on paid plans. Directus's licence is free only for small organisations. Wagtail would mean a move to Python. Keystone has no drafts.

Why Payload 4 rather than 3? Hierarchy for areas, crash-safe jobs, authorship, dashboard widgets and per-language publishing all replace code we would otherwise write. The cost is that there is no route back to version 3.

Why must the core run alone? A system that needs outside services cannot be tested automatically, cannot be handed to a new developer, and stops when a vendor changes its terms.

Why refuse to start when a port has no adapter? A deployment with nowhere to verify claims is broken, and whoever deploys it should find out first.

Why keep every plugin's fields in the schema? Postgres needs a migration for every column. One schema means one migration history, and turning a plugin on never needs a database change.

Why editor-defined ratings? New claim types need their own scales. A public statement is accurate or not, never "kept".

Why derive a claim's rating? A stored rating can drift from the evidence. A derived one always matches the latest update.

Why publish per language? A translation is often finished later than the original. Half a French page in English is a fault, not a convenience.

Why does a source document sit above its claims? The document is the evidence. Withdrawing a fake manifesto should take one action.

Why check quotes against the source? An AI model can produce a convincing promise that nobody made. One text search removes that risk.

Why keep the claim in PromiseTracker when verifying elsewhere? Version 2 kept promises in Check and rebuilt them from its reports, so a change in Check could remove one. Taking only the verdict

back means no outside system can.

Why seat addresses? People know where they live, not who represents them. A seat address is a stable link to "the governor of Nairobi", whoever that is.

Why an SDK between the site and the platform? It lets the site run in the same process at launch and on its own later, and lets a partner replace any part of the site without touching the platform.

21. Appendix A: items to confirm

Meedan Check, version 1

1. The workspace's status list, and the verdict each maps to.
2. That the workspace can send webhooks to an outside address, and which events it sends.
3. The report state names, `published` and `paused`, in the current Check version.

Infrastructure

4. A managed Postgres provider that offers `unaccent` and `pg_trgm`.
5. Where the Docker image runs.
6. The email provider for notifications and newsletter confirmations.
7. Editor sign-in: passwords only, or single sign-on with Code for Africa accounts, and whether super-admins need a second factor.
8. The analytics tool, if any, for version 1.
9. The volume at which Postgres full-text search stops being enough.

Superdesk, later

10. The Content API prefix, whether the Production API runs, the content profile, the vocabulary and desk ids, a service account, whether HTTP push is available, and the Superdesk version.

Maps, version 2

11. The map tile provider and its licence, and the source of boundary data and its attribution rules.

22. Appendix B: what v2 does today

Problems v3 fixes

Problem	Where to see it in v2
Airtable is the source of documents and the work queue	Six pipeline tasks write status text into Airtable cells
Only the Check sync creates promises	`promises.meedanId` is required and unique in `src/collections/Promises.ts`
No promise history	The timeline component receives an empty list of events
Nothing is cached	Every public route sets `force-dynamic`
The promise list loads everything	`pagination: false` in `src/lib/data/promiseQueries.ts`
Search does nothing	`src/components/Search.tsx` has no handler
Three publishing conventions	Two yes-or-no fields and one free-text field, `promises.publishStatus`
Tenant isolation is not enforced	`userHasAccessToAllTenants: () => true` in `src/plugins/index.ts`
Accented characters are removed	`cleanText` in `src/tasks/extractDocuments.ts`

Code v3 keeps

What	Where in v2
Safeguards for fetching outside addresses	`src/utils/ssrf.ts` , `src/utils/files.ts`
File type checking	`src/utils/fileSignature.ts`
The AI quote check	`src/tasks/extractPromises.ts`
The AI vendor list	`src/lib/ai/providerCatalog.ts` , `src/lib/ai/providerRegistry.ts`
The Check API client	`src/lib/meedan.ts`
Webhook duplicate detection	`src/app/api/meedan-sync/route.ts`
The encrypted settings field	`src/fields/encryptedSecret.ts`
The security tests	`tests/int/`

The scheduled job lock in `src/lib/workflowLocks.ts` is no longer needed, because Payload 4's job concurrency keys do the same job.

From the redesign

The redesign repository, `CodeForAfrica/promisetracker_redesign` , supplies the components, page layouts, the counts-not-percentages rule, the archived official layout, the rating history page, the update

form and the share images. Its data, now in static files, moves behind the SDK, and its hardcoded ratings come from the rating scales instead.